From a Latent-Variable Model to a Trainable Generator

N-20251009

A latent-variable model has a simple forward recipe: sample a hidden code z, and then sample an observation x given that code. Even a mixture of simple conditional Gaussian laws can produce several peaks after the hidden cause is forgotten; determining which cause explains an observation then requires Bayes' rule. With images, the remaining questions are practical: what exactly does a neural decoder output, how do we measure an entire dataset under the model, and why do we need a recognition network when we can sample from the model already?[1]

§1.1 A decoder produces probabilities, not just pixels

For a latent code $z \in \mathbb{R}^d$, the standard VAE starts with a simple prior such as

$$p(z) = \mathcal{N}(z; 0, I).$$

A neural decoder maps z to the parameters of a conditional distribution $p_\theta(x \mid z)$. The model is

$$\boxed{p_\theta(x, z) = p(z)\, p_\theta(x \mid z).}$$

The direction $z \to x$ is the generative direction; unlike a reconstruction-only autoencoder, it can begin from a random latent draw without any observed image.

The observation distribution must match the data. If an image consists of binary pixels $x_j \in \{0, 1\}$, the decoder might output a number $\pi_j(z) \in (0, 1)$ for each pixel and define

$$p_\theta(x \mid z) = \prod_{j=1}^{D} \pi_j(z)^{x_j} (1 - \pi_j(z))^{1 - x_j}.$$

Each output is the probability that a pixel is on, not the pixel itself. The negative log-likelihood becomes

$$-\log p_\theta(x \mid z) = -\sum_j [x_j \log \pi_j(z) + (1 - x_j) \log(1 - \pi_j(z))],$$

which is binary cross-entropy.

For a real-valued feature vector, a Gaussian decoder might be preferable:

$$p_\theta(x \mid z) = \mathcal{N}(x; \mu_\theta(z), \sigma_x^2 I).$$

Up to terms independent of θ, the negative log-likelihood is

$$-\log p_\theta(x \mid z) = \frac{\|x - \mu_\theta(z)\|^2}{2\sigma_x^2} + \text{constant}.$$

So the familiar squared reconstruction error is a probabilistic modeling choice, with the noise variance determining its scale. The likelihood specification matters as much as the neural-network architecture.

[1] The model architecture and approximate-inference viewpoint follow Kingma and Welling, *An Introduction to Variational Autoencoders* (2019), and C. Doersch, *Tutorial on Variational Autoencoders*.

§1.2 Conditional Independence and Correlation

The product over pixels assumes that the pixels are independent *given* z. That is a computational simplification. It does not imply that pixels are independent in the generated dataset, because the same latent variable affects every pixel.

A tiny example makes the distinction sharp. Let Z be a fair bit. Given $Z = z$, set $X_1 = z$ and $X_2 = z$ deterministically. The two observations are trivially independent conditional on Z, but marginally

$$\Pr(X_1 = 1) = \Pr(X_2 = 1) = \frac{1}{2}, \qquad \Pr(X_1 = 1, X_2 = 1) = \frac{1}{2} \neq \frac{1}{4}.$$

Thus the shared latent cause has created dependence between the observed variables. This is why even a decoder using a factorized pixel likelihood can model correlated images after latent codes are marginalized out.

The price of a simple factorized decoder is also visible. Conditioned on a *fixed* code z, it cannot represent unexplained correlations between pixels beyond those already encoded by z. A better latent representation or a richer likelihood may be needed when the data exhibit residual structure.

§1.3 A dataset means many hidden causes

For observations $x^{(1)}, \ldots, x^{(N)}$, assume independent examples and give each example its own latent variable $z^{(i)}$. The joint law is

$$p_\theta(x^{(1:N)}, z^{(1:N)}) = \prod_{i=1}^{N} p(z^{(i)}) p_\theta(x^{(i)} \mid z^{(i)}).$$

The same prior and decoder are used for every observation, while each observation has its own latent code. A small continuous example also reveals the geometry of the marginal distribution produced by a shared latent cause.

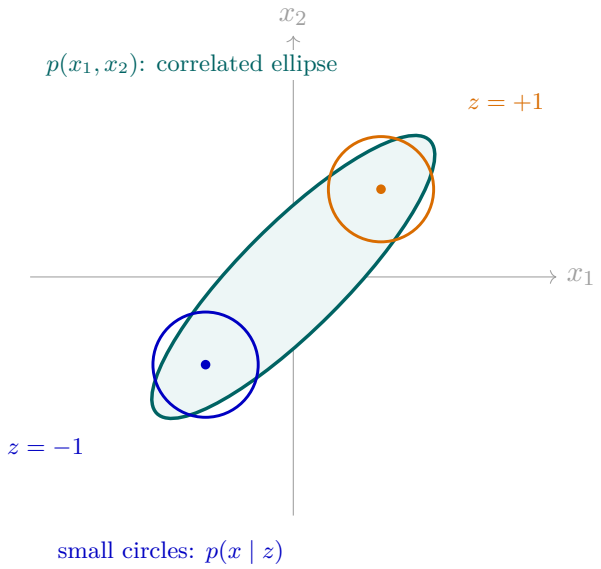


Figure 1.1: A one-dimensional latent $Z \sim \mathcal{N}(0,1)$ generates $X_1 = Z + \varepsilon_1$ and $X_2 = Z + \varepsilon_2$, where the independent noises have variance 0.4^2. Each conditional distribution $p(x \mid z)$ is a circular Gaussian (two examples are outlined). After forgetting z, $p(x)$ has covariance $\left(\begin{smallmatrix} 1.16 & 1 \\ 1 & 1.16 \end{smallmatrix}\right)$ and elongated elliptical contours. Conditional independence has produced strong marginal correlation. Ellipses mark equal Mahalanobis radii.

Learning by marginal maximum likelihood would maximize

$$\log p_\theta(x^{(1:N)}) = \sum_{i=1}^{N} \log p_\theta(x^{(i)}),$$
$$p_\theta(x) = \int p(z) p_\theta(x \mid z)\, dz.$$

The integral is the obstacle. Even if the decoder evaluates quickly for one z, integrating its output over all plausible latent codes can be very expensive.

§1.4 When Prior Sampling Fails

The identity

$$p_\theta(x) = \mathbb{E}_{z \sim p(z)}[p_\theta(x \mid z)]$$

suggests a Monte Carlo approximation by averaging $p_\theta(x \mid z)$ over prior samples. This works in principle as an unbiased estimator of the likelihood, but it can be practically useless when the observation is explained by only a tiny part of latent space.

Here is an exact discrete example. Take K independent fair latent bits, $Z = (Z_1, \ldots, Z_K)$. Let the observed bit X equal one exactly when every hidden bit equals one:

$$\Pr(X = 1 \mid Z = z) = \begin{cases} 1, & z = (1, \ldots, 1), \\ 0, & \text{otherwise.} \end{cases}$$

Generating a sample is trivial. Yet

$$\boxed{\Pr(X = 1) = 2^{-K}.}$$

If we draw M latent samples from the prior and average their likelihoods for the observed event $X = 1$, the estimate is exactly zero unless at least one sample hits the all-ones configuration. The probability of never hitting it is

$$\boxed{\left(1 - 2^{-K}\right)^{M}}.$$

For $K = 20$, even ten thousand prior samples miss the relevant latent state with probability about 0.9905. To obtain a reasonable probability of observing even one relevant sample, M must be of order 2^K.

Once $X = 1$ is known, however, the posterior is concentrated on the all-ones code. The difficulty was not evaluating the decoder; it was sampling latent states without regard to the observed data. This is the basic reason approximate posterior inference is so valuable.

§1.5 Recognition is a fast approximate inverse

The recognition network supplies

$$q_\phi(z \mid x) \approx p_\theta(z \mid x).$$

For a continuous latent code, a common choice is a diagonal Gaussian,

$$q_\phi(z \mid x) = \mathcal{N}\big(z; \mu_\phi(x), \mathrm{diag}(\sigma^2_\phi(x))\big).$$

The neural encoder computes $\mu_\phi(x)$ and $\sigma_\phi(x)$ from the input image, describing a whole distribution of plausible codes. The decoder and encoder answer different conditional questions:

$$\underbrace{p_\theta(x \mid z)}_{\text{what could this cause generate?}} \quad \text{and} \quad \underbrace{q_\phi(z \mid x)}_{\text{what might have caused this observation?}}.$$

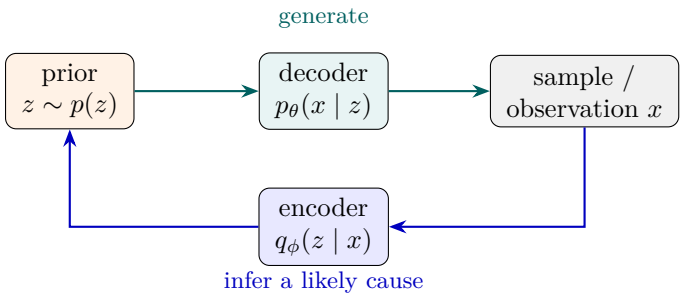


Figure 1.2: The probabilistic decoder defines the forward model. The recognition network approximates the reverse conditional distribution without computing its exact normalizing integral.

An ordinary variational method could optimize a separate approximate posterior for every training example. A VAE amortizes that computation: the same encoder parameters ϕ map every new input to posterior parameters in a single forward pass. This makes large-dataset inference fast, but it constrains all approximations to come from one shared learned function.

There are now two distinct approximation errors to keep in mind. A diagonal Gaussian family may be unable to represent a multimodal posterior, and even within that family a learned encoder may not output the best variational parameters for every example. The evidence lower bound addresses the training objective and quantifies the discrepancy with the true posterior; the next mathematical step is to optimize that bound with gradients.